

Correlation-First Blame Propagation: Personalized PageRank and Parameter-Free Change-Point Detection as Pre-LLM Graph Priors for Root-Cause Analysis

Victor Robin
BlueRobin (independent homelab research)

June 2026

Abstract

This paper describes the deterministic, training-free pre-LLM scoring layer of the BlueRobin Debug Agent, an advisory root-cause-analysis (RCA) system for a self-hosted homelab operated under a hard ≈ 50 EUR/month budget. Before any large-language-model (LLM) tokens are spent, the system seeds the agent’s hypothesis space with a ranked candidate list produced entirely by classical, parameter-free graph and time-series statistics. The core is a four-stage pipeline: (1) an error-rate-weighted *personalized PageRank* over the runtime dependency graph that propagates “blame” from the alerting (symptom) service to the dependencies it transitively calls; (2) a Bayesian repeat-offender history prior; (3) a parameter-free Bayesian Online Change-Point Detection (BOCPD) onset detector paired with a nonparametric, median-based robust scorer in the style of BARO [Pham et al., 2024]; and (4) a five-term re-normalizing additive blend ($w_{\text{topo}}=0.35$, $w_{\text{tele}}=0.25$, $w_{\text{hist}}=0.15$, $w_{\text{chg}}=0.15$, $w_{\text{baro}}=0.10$) that provably collapses byte-for-byte to a topology/telemetry/history baseline when any lane is absent. The design makes an explicit, repeatedly enforced architectural choice: it uses *correlation and change-point* signals only and rejects causal discovery (PC/FCI/Granger/learned causal graphs), a decision validated against two independent re-evaluations of the RCA literature [Pham et al., 2025, Zhang et al., 2024] and policed by a CI reflection scan over the scoring assembly. We report results on a self-seeded nine-incident benchmark with the explicit caveat—shared by the authors—that it is an oversimplified benchmark and that the reported MTTR figures are sub-second replay-clock proxies, not production mean-time-to-resolution.

1 Introduction

Modern microservice RCA increasingly reaches for an LLM agent: a ReAct-style [Yao et al., 2022] loop that calls observability tools, reads traces and logs, and narrates a hypothesis. The honest published ceiling for *autonomous* LLM RCA is low—the best agent on OpenRCA [Xu et al., 2025] reaches roughly 11.3% exact-match, and the best frontier model on ITBench-AA [IBM Research and Artificial Analysis, 2026] roughly 47% precision-at-full-recall. LLM-only RCA is also structurally fragile: it lacks grounding in system topology, it hallucinates plausible-but-wrong causes, and—per the MAST failure taxonomy [Cemri et al., 2025]—its dominant failure modes are *incorrect self-verification* and *premature termination*. An agent dropped cold into a tool loop wastes its turn budget rediscovering the obvious shape of the system on every incident.

The pre-LLM scoring layer described here addresses the *cold-start* problem directly. Rather than asking the model to find the needle from scratch, the system computes a ranked candidate list deterministically and injects it into the prompt as “prioritised leads to confirm, not conclusions.”

The leads come from cheap, well-understood statistics that independent benchmarks show *beat* the expensive causal-discovery machinery on real data. The LLM’s job is then verification and narration, not blind search.

The contributions of this paper are:

- **An error-weighted personalized PageRank “blame propagation” pre-ranker** (`PageRank.cs`, `RootCauseRanker.cs`) that seeds the teleport vector at the symptom service and amplifies blame flow along high-error dependency edges—a power-iteration computation with no external dependency, fully deterministic, computed in microseconds on a homelab-sized graph.
- **A parameter-free BOCPD onset detector plus a nonparametric robust scorer** (`BaroAnomalyScorer.cs`) that reproduces the BARO recipe in dependency-free C#: no hazard knob, no training, no GPU, total on degenerate input.
- **A five-term re-normalizing additive blend** (`BaroBlendScorer.cs`) with a *provable fail-open collapse*: when the change lane and/or BARO lane are absent, the blend reduces byte-for-byte to a two- or three-term baseline, so the ranker degrades gracefully rather than failing closed.
- **An explicit, CI-enforced rejection of causal discovery.** The scoring assembly carries no PC/FCI/Granger/causal-graph code, and a reflection scan (TS-57) fails the build if a causal symbol appears. We justify this against the literature rather than asserting it.
- **A budget-first systems posture.** The entire layer runs inside an already-deployed 2 GiB FalkorDB instance with zero new always-on workload, consistent with the platform’s ≈ 50 EUR/month ceiling.

This is a systems / experience report, not an empirical breakthrough. The layer is advisory by design—a human always merges—and the evaluation is deliberately modest.

2 Related Work

Classical and statistical RCA. Spectrum-and-random-walk methods rank suspect services by propagating anomaly mass over a dependency graph. MicroRank [Yu et al., 2021] combines spectrum analysis with PageRank-style blame propagation; TraceRCA [Li et al., 2021] mines suspicious sets from anomalous traces and ranks by conditional probability. BARO [Pham et al., 2024] detects the failure *onset* with multivariate BOCPD and then ranks with a nonparametric RobustScorer; it is parameter-free, training-free, and runs in roughly 0.01 s. Our pre-LLM layer sits squarely in this family: error-weighted personalized PageRank for propagation, plus a hand-rolled BARO core for onset and robust ranking.

Causal RCA, and why we decline it. A parallel line of work learns a causal graph and infers the root from it: CIRCA [Li et al., 2022] frames a fault as an intervention and applies a regression-based hypothesis test; RCD [Ikram et al., 2022] does localized causal discovery with a ψ -PC variant; RUN [Lin et al., 2024] combines neural Granger causality with personalized PageRank. The decisive evidence against adopting causal *discovery* in our setting comes from two independent re-evaluations. RCAEval [Pham et al., 2025] re-runs 15 baselines across 735 cases and finds correlation methods (BARO Avg@5 0.80, TraceRCA 0.77) decisively beating causal ones (CIRCA 0.46, MicroCause 0.20, RCD 0.13). “Root Cause Analysis with LLMs...How Far Are We from

Causal Inference?” [Zhang et al., 2024] finds PC/FCI/Granger full-graph methods are “no better or only slightly better than random,” require the *exact* failure time (a 60 s error degrades them significantly), and blow up computationally (PC+KCI exceeding an hour per case at ten nodes versus BARO’s ≈ 0.01 s). The defensible causal *idea*—“fault as intervention”—survives in our change-correlation term; the *graph-learning step* is the weak, expensive link we drop.

Anomaly detection. TSB-AD [Liu and Paparrizos, 2024] benchmarks 40 algorithms over 1,070 series and concludes that “simpler architectures and statistical methods often yield better performance”; foundation models neither win nor pay for their 10–50 $\times$ slowdown. This is the empirical warrant for a median-based, distribution-free robust scorer over a learned detector.

Benchmarks. OpenRCA [Xu et al., 2025], AIOpsLab [Chen et al., 2025], ITBench [IBM Research, 2025] and RCAEval [Pham et al., 2025] supply the metric vocabulary (AC@k, Avg@k, MRR, MTDD/MTTR/TTM) we adopt internally. RCAEval’s AC@k/Avg@k/MRR is the closest analogue to our own scorecard. A standing caveat [Anonymous, 2025] warns that current RCA benchmarks are oversimplified and overstate real capability—a caveat we apply to our own nine-incident set in Section 6.

3 Method / Architecture

The pre-LLM layer transforms an incoming alert into a ranked list of candidate root causes using only deterministic statistics. Figure 1 shows where it sits.

3.1 Personalized PageRank blame propagation

The substrate is the runtime dependency graph: `Service-[:CALLS|READS_FROM]->Service` edges in the persistent `bluerobin_stack` FalkorDB graph, each carrying a measured `error_rate` from Tempo service-graph metrics. `RootCauseRanker.RankAsync` (per **FEAT-027** / **FR-SG-6**, the RCA query API requirement) fetches the *whole* runtime graph—it is small, tens of edges on a homelab—with a single parameterless Cypher template, never caller-interpolated (**SEC-SG-4**, parameterized Cypher only):

```
MATCH (a:Service)-[e:CALLS|READS_FROM]->(b:Service)
RETURN a.name, b.name, coalesce(e.error_rate, 0.0)
```

Each directed edge is weighted

$$w(\text{from} \rightarrow \text{to}) = \text{BaseEdgeWeight}(0.1) + \text{error_rate}. \quad (1)$$

The base weight keeps the graph connected so blame can flow at all; the additive `error_rate` term amplifies flow toward failing dependencies. The ranker then runs weighted *personalized* PageRank (`PageRank.Run`), where the personalization (teleport) vector is a point mass at the symptom service:

```
teleport    = { [symptomService] = 1.0 }    // all other nodes 0
damping     = 0.85
iterations  = 50    // fixed -> deterministic, no tolerance nondeterminism
```

Power iteration is initialized at the teleport distribution. At every step, mass is mixed $(1 - d)$ toward the teleport vector and d along row-normalized out-edges; dangling nodes (no usable out-edge) redistribute their mass through the teleport vector. Because all blame re-injects at the symptom, score concentrates on the dependencies the symptom transitively calls, weighted by how error-prone those paths are. This is “blame propagation”: the symptom is innocent of *being* the cause but is the seed from which suspicion flows downstream. The implementation is pure and dependency-free, returns a distribution summing to ≈ 1 , and is fully unit-tested. Per-node anomaly teleport (seeding mass at *every* anomalous service, not just the symptom) is noted in the code as a deliberate future refinement; today the teleport is symptom-only.

3.2 Bayesian repeat-offender history prior

PageRank scores for a candidate c are folded with a multiplicative history prior:

$$\text{score}'(c) = \text{score}(c) \cdot (1 + \text{HistoryInfluence}(0.5) \cdot \text{prior}(c)), \quad (2)$$

where $\text{prior}(c) \in [0, 1]$ is a Bayesian repeat-offender estimate drawn from the graph’s RCA write-back history (ROOT_CAUSED_BY edges from past validated incidents). A service that has been the validated root cause before is, all else equal, a more credible candidate—a cheap Bayesian shrinkage toward known offenders, capped at a $1.5\times$ boost so history can never dominate live error signal.

3.3 Parameter-free BOCPD onset detection and the robust scorer

The change-point lane (`BaroAnomalyScorer.cs`, per **FR-SG-30** and **ADR-037 D5**) reproduces BARO in dependency-free C#. It has three pieces.

BocpdChangePoint.DetectOnset. A parameter-free retrospective change-point detector over a single KPI series. It sweeps every candidate boundary k and chooses the one that maximizes the two-segment Gaussian log-likelihood gain over the no-change (single-segment) model, each segment scored under its own MLE mean and variance (the offline reduction of a Normal–Gamma conjugate posterior). Crucially, *no hazard or threshold knob is passed in*: the significance test is a BIC-style penalty derived from series length,

$$\text{onset} = \arg \max_k [\text{LL}(0, k) + \text{LL}(k, n) - \text{LL}(0, n)] \quad \text{if gain} > \log(n), \text{ else null}, \quad (3)$$

so a flat or stationary series yields *no* onset—the fail-open sentinel that prevents fabricating a change-point where none exists. A variance floor keeps a perfect step finite. A minimum segment length of 3 guarantees each side has a defined mean and variance.

RobustScorer. The nonparametric BARO scorer: the *robust relative magnitude* of the post-onset level shift, computed median-based and scale-free across heterogeneous KPIs, with **no Gaussian/z-score assumption**. Bounded to $[0, 1]$; exactly 0 when there is no onset or no shift.

BaroAnomalyScorer. The façade the ranker consumes. Given a service’s three Tempo service-graph KPI series (`ServiceKpiSeries`: call-rate, error-ratio, p95-latency), it returns the maximum robust shift across those KPIs as a bounded `baro` sub-score (multivariate by max-pooling; $0 \Rightarrow$ no boost).

The whole core is pure, dependency-free, and total on empty/short input, mirroring the existing `AnomalyStatistics` discipline.

3.4 The five-term re-normalizing additive blend

BaroBlendScorer (per **FR-SG-30**, **NFR-SG-2**, **ADR-037 §T5**) fuses five deterministic sub-scores into one ranking score. The blend evolved across increments; the canonical five-term form and its weights are given in Table 1.

Table 1: The canonical five-term blend.

Term	Weight	Source signal
topo	0.35	topology criticality of the candidate service
tele	0.25	telemetry: measured error rate on the dependency edge (halved if the edge is stale)
hist	0.15	repeat-offender history prior (§3.2)
chg	0.15	onset-window change recency (Deploy/Change near the onset)
baro	0.10	BARO robust change-point sub-score (§3.3)

The weights sum to 1.0. The change term $\text{chg}(c)$ is the maximum over in-window Deploy/Change events of a recency ramp $1 - ((T - t_s)/W)$ over a half-open onset window $[T - W, T]$ with $W = 1800$ s—privileging the deploy or commit nearest the failure onset, the single strongest signal in the SRE literature (Google’s “ $\approx 70\%$ of outages are change-induced” [Beyer et al., 2016]).

The **fail-open collapse** is the load-bearing guarantee (**NFR-SG-2**). When a lane’s signal is absent—no Deploy/Change in window, or a KPI series too short for BOCPD—the blend *re-normalizes over the present terms only* rather than penalizing the candidate with a zero. Concretely:

- All five present $\rightarrow 0.35/0.25/0.15/0.15/0.10$.
- BARO absent, change present $\rightarrow$ the four-term blend $0.40/0.30/0.15/0.15$.
- Change and BARO both absent $\rightarrow$ the three-term baseline $0.50/0.35/0.15$ (topo/tele/hist).

Each collapse is *byte-for-byte identical* to the earlier-increment scorer it reduces to—the four-term **OnsetWindowChangeScorer** delegates to the three-term **RootCauseScorer** when the change lane is empty, and the five-term **BaroBlendScorer** delegates to the four-term form when BARO is empty. This is what lets the layer ship a richer ranker without risking a regression: with the new lanes off, the output is provably the old output. Ties break deterministically (score descending, then **NodeId** ascending)—a fix for an early flaky AC@1 caused by FalkorDB returning rows in nondeterministic order.

3.5 The deliberate rejection of causal discovery

The scoring assembly is annotated and tested as “*correlation + change-point only—explicitly NO causal discovery (PC/FCI/Granger/causal-graph).*” This is not an omission; it is an enforced invariant. A reflection scan (TS-57) runs over the analysis assembly in CI and fails the build if any causal-discovery symbol is present. The justification is the Section 2 evidence: on independent benchmarks, correlation and change-point beat learned causal graphs, while causal discovery demands exact fault timing and prohibitive compute. The “fault-as-intervention” framing we *do* keep—but it lives entirely in the change-correlation term (a deploy in the onset window *is* the intervention), not in any structure-learning step.

4 Implementation

The layer is C# (.NET 10), part of the Debug Agent service (FEAT-008 / FEAT-027). Key types are listed in Table 2.

Table 2: Implementation components.

Component	File	Role
PageRank	Analysis/PageRank.cs	pure weighted personalization
RootCauseRanker	Analysis/RootCauseRanker.cs	fetches the runtime graph
BocpdChangePoint	Analysis/BaroAnomalyScorer.cs	parameter-free BOCPD
RobustScorer / BaroAnomalyScorer	Analysis/BaroAnomalyScorer.cs	nonparametric robust shi
OnsetWindowChangeScorer	StackGraph/OnsetWindowChangeScorer.cs	four-term blend (0.40/0.3
BaroBlendScorer	StackGraph/BaroBlendScorer.cs	five-term blend (0.35/0.2

Graph store. The runtime dependency graph lives in the existing FalkorDB instance (`falkordb/falkordb:v4.4`, Redis wire protocol on port 6379, 2 GiB memory limit) as a second graph key `bluerobin_stack`, queried with parameterized OpenCypher via `GRAPH.QUERY`. No second time-series database and no new always-on workload is introduced; the `bluerobin` node is memory-saturated, which is the binding constraint behind reusing the deployed FalkorDB rather than standing up a dedicated graph engine.

Telemetry source. Edge `error_rate` and the per-service KPI series come from Tempo’s metrics-generator service graphs (`traces_service_graph_request_*`)—the dependency graph and the RED signals are read from the same observability plane the cluster already runs. Change/Deploy nodes are ingested from Flux reconciliations and Git commits.

Cost. The entire pre-LLM layer consumes *zero* LLM tokens: PageRank, BOCPD, the robust scorer, and the blend are all CPU-only arithmetic over a tens-of-edges graph and short KPI windows, completing in well under a millisecond. This is what makes a graph-grounded RCA stack viable under the platform’s hard ceiling—Hetzner worker $\approx$ EUR 17.5/mo, Cloudflare R2/Tunnel free tier, and an LLM budget capped at EUR 20/mo (COST-7), for $\approx$ 50 EUR/month total. The expensive alternative—GraphRAG-style community indexing at roughly \$20–40 per 1M tokens, or causal discovery at >1 hr/case—is explicitly out of budget. Spending the LLM budget only on *verifying* a deterministically-computed shortlist is the core FinOps move.

The error-weighted personalized-PageRank seed is abridged as:

```
// RootCauseRanker.RankAsync -- error-weighted personalized PageRank seed
var weight = BaseEdgeWeight + (err > 0 ? err : 0);           // 0.1 + error_rate
var scores = PageRank.Run(
    nodes, outEdges,
    teleport: new Dictionary<string, double> { [symptomService] = 1.0 },
    damping: 0.85, iterations: 50);
// fold the repeat-offender prior: score * (1 + 0.5 * prior[node])
```

The ranker is advisory and best-effort by contract (**NFR-SG-2**): an empty, stale, or unreachable graph yields no candidates and never throws—the agent then proceeds on its cold tool-floor.

This fail-open posture is the same one the blend’s re-normalization enforces at the scoring level: every degradation path returns a valid (possibly empty) ranking rather than an error.

5 Evaluation

The pre-LLM ranker is exercised by a deterministic offline replay harness (`RcaEvalHarness`, per **FR-SG-23** / **ADR-036**) that replays a labelled ground-truth incident set through the *real* ranking path over a seeded FalkorDB and emits an AC@1 + MTTR scorecard. The set is **nine self-seeded labelled incidents** spanning three fault classes (auth/login, data/datastore, deploy/config), drawn from existing synthetic faults—no ChaosMesh—so the set is not trivially gameable.

The harness has two arms. The **scorer-only** arm plants the labelled cause as an unambiguous top-1, so its AC@1 = 1.0 *by construction*—a regression sentinel, not an accuracy measure. The honest arm is `ReplayLivePathAsync`, the live-path arm, which derives onset from the alert and **injects distractor candidates** (a deliberate topology trap: distractor criticality 0.95 *above* the true cause’s 0.9, distractor error-rate 0.55) so that AC@1 < 1.0 is achievable. Results are shown in Table 3.

Table 3: Replay-harness scorecards on the nine-incident self-seeded set. MTTR is a sub-second replay-clock proxy, not production mean-time-to-resolution.

Arm	AC@1	Avg@k	MRR	MTTR (s, replay-clock)	Incidents
Scorer-only baseline (1.0 by construction)	1.0	1.0	1.0	0.0017	9
Live-path baseline (distractors injected)	1.0	1.0	1.0	0.0032	9

On the nine-incident set, the ranker localizes the labelled root cause at top-1 in all nine incidents *even with distractors injected*. Avg@k and MRR reduce to AC@1 here because each incident carries a single labelled cause scored at top-1 (per ADR-036 D1; extended when multi-candidate labels land). The per-commit merge gate fails any candidate that drops AC@1 below 0.98 or inflates MTTR by more than 30 s (`GateTolerance(AcAt1Drop: 0.02, MttrSecondsIncrease: 30.0)`).

Ablations. Because the five-term blend collapses byte-for-byte when lanes are disabled, the harness runs *shadow-vs-active* ablations: with the BARO/change lanes in shadow mode the scorecard is identical to the baseline (the “feature off” control), and the two change-induced incidents (GT-03 bad-deploy, GT-07 bad-commit) must keep AC@1 = 1.0 in every active-mode run—a guard that the new lanes never *suppress* a true positive they should have surfaced. There is no graph-vs-no-graph ablation as a graded metric: the graph is the substrate of the entire RCA path, and “no graph” surfaces only as the three-term fail-open floor.

6 Limitations & Threats to Validity

We state these plainly; the authors themselves call the benchmark oversimplified.

- **The benchmark is small and self-seeded.** Nine incidents, three fault classes, labelled by the same authors who built the system. AC@1 = 1.0 on this set is *not* evidence of production accuracy. It is a regression sentinel and a sanity check, nothing more. External yardsticks (OpenRCA $\approx$ 11%, ITBench-AA $\approx$ 47%) are referenced as context, never as something this set is comparable to.

- **MTTR is a replay-clock proxy.** The reported sub-second MTTR figures are the wall-clock around an in-process replay, not time-to-resolution of a real incident. Production MTTD/MTTA/MTTR are computed separately from live lifecycle data (SLO-15/SLO-16), but no measured production value is claimed here.
- **The distractor trap is hand-designed.** The live-path arm’s distractors are deliberately adversarial on topology but were authored by us; a different distractor distribution could lower AC@1. We make no claim that the trap is representative of real misleading topology.
- **Symptom-only teleport.** PageRank seeds blame solely at the alerting service. Multi-symptom incidents, where several services are independently anomalous, are not yet modeled by per-node anomaly teleport (a noted future refinement). Single-root-cause is an assumption baked into the current scorer.
- **Correlation is not causation—by design.** We deliberately do *not* attempt causal discovery, so the ranker can be fooled by a confounded correlation (e.g. a shared upstream that fails two services at once). The change-correlation term mitigates but does not eliminate this; the human-in-the-loop merge is the backstop.
- **Advisory only.** Nothing auto-remediates. Every output is a lead for a human reviewer. This matches the state of the art for autonomous RCA rather than conceding a limitation—but it does mean the layer’s value is measured by how well it *focuses* a human, which this benchmark does not capture.

7 Conclusion

The pre-LLM scoring layer demonstrates that a large fraction of the RCA problem can be handled *before* the LLM ever runs, using deterministic, training-free statistics that the independent literature shows outperform expensive causal discovery. Error-weighted personalized PageRank propagates blame from symptom to suspect; a parameter-free BOCPD onset detector and a nonparametric robust scorer reproduce BARO with no training and no GPU; and a five-term re-normalizing blend fuses these with a provable fail-open collapse to a baseline. The whole layer runs in microseconds inside an already-deployed FalkorDB, consuming no LLM tokens, which is precisely what makes a graph-grounded agentic RCA stack feasible at ≈ 50 EUR/month. The headline is not an accuracy number—our benchmark is too small and too friendly for that—but an architecture: spend deterministic compute to build a shortlist, spend scarce LLM tokens only to verify it, and refuse, on the evidence, to learn a causal graph.

References

- Anonymous. Rethinking the evaluation of root cause analysis: A fault-propagation-aware benchmark. *arXiv preprint*, 2025. URL <https://arxiv.org/abs/2510.04711>. arXiv:2510.04711.
- Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O’Reilly Media, 2016. URL <https://sre.google/sre-book/introduction/>.
- Mert Cemri et al. Why do multi-agent LLM systems fail? the MAST failure taxonomy. IBM Research and UC Berkeley, 2025. URL <https://huggingface.co/blog/ibm-research/itbenchandmast>.

- Yinfang Chen et al. AIOpsLab: A holistic framework to evaluate AI agents for enabling autonomous clouds. *arXiv preprint*, 2025. URL <https://arxiv.org/abs/2501.06706>. arXiv:2501.06706.
- IBM Research. ITBench: A benchmark for evaluating AI agents on IT automation tasks. *arXiv preprint*, 2025. URL <https://arxiv.org/abs/2502.05352>. arXiv:2502.05352.
- IBM Research and Artificial Analysis. ITBench-AA: Agentic SRE root-cause analysis benchmark. Artificial Analysis and IBM Research, 2026. URL <https://huggingface.co/blog/ibm-research/itbench-aa>.
- Azam Ikram, Sarthak Chakraborty, Subrata Mitra, Shiv Saini, Saurabh Bagchi, and Murat Kocaoglu. Root cause analysis of failures in microservices through causal discovery (RCD). In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL <https://openreview.net/forum?id=weoLjoYFvXY>.
- Mingjie Li, Zeyan Li, Kanglin Yin, Xiaohui Nie, Wenchi Zhang, Kaixin Sui, and Dan Pei. Causal inference-based root cause analysis for online service systems with intervention recognition (CIRCA). *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2022. URL <https://arxiv.org/abs/2206.05871>. arXiv:2206.05871.
- Zeyan Li, Junjie Chen, Rui Jiao, Nengwen Zhao, Zhijun Wang, Shuwei Zhang, Yanjun Wu, Long Jiang, Leiqin Yan, Zikai Wang, Zhekang Chen, Wenchi Zhang, Xiaohui Nie, Kaixin Sui, and Dan Pei. Practical root cause localization for microservice systems via trace analysis (TraceRCA). In *Proceedings of the IEEE/ACM 29th International Symposium on Quality of Service (IWQoS)*, 2021. doi: 10.1109/IWQOS52092.2021.9521340. URL <https://ieeexplore.ieee.org/document/9521340/>.
- Cheng-Ming Lin, Ching Zhang, Tom Lin, Wei Cheng, Pei-Hua Ji, and Cho-Jui Hsieh. Root cause analysis in microservice using neural granger causal discovery (RUN). *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024. URL <https://arxiv.org/abs/2402.01140>. arXiv:2402.01140.
- Qinghua Liu and John Paparrizos. The elephant in the room: Towards a reliable time-series anomaly detection benchmark (TSB-AD). In *Proceedings of the VLDB Endowment*, 2024. URL <https://github.com/thedatumorg/TSB-AD>.
- Luan Pham, Huong Ha, and Hongyu Zhang. BARO: Robust root cause analysis of microservices via multivariate Bayesian online change point detection. *Proceedings of the ACM on Software Engineering (FSE)*, 2024. URL <https://arxiv.org/abs/2405.09330>. arXiv:2405.09330.
- Luan Pham, Huong Ha, and Hongyu Zhang. RCAEval: A benchmark for root cause analysis of microservice systems with telemetry data. *Proceedings of the ACM Web Conference (WWW)*, 2025. URL <https://arxiv.org/abs/2412.17015>. arXiv:2412.17015.
- Junjielong Xu et al. OpenRCA: Can large language models locate the root cause of software failures? International Conference on Learning Representations (ICLR), 2025. URL <https://github.com/microsoft/OpenRCA>.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint*, 2022. URL <https://arxiv.org/abs/2210.03629>. arXiv:2210.03629.

Guangba Yu, Pengfei Chen, Hongyang Chen, Zijie Guan, Zicheng Huang, Linxiao Jing, Tianjun Weng, Xinmeng Sun, and Xiaoyun Li. MicroRank: End-to-end latency issue localization with extended spectrum analysis in microservice environments. In *Proceedings of the Web Conference (WWW)*, 2021. doi: 10.1145/3442381.3449905. URL <https://dl.acm.org/doi/10.1145/3442381.3449905>.

Cheryl Zhang et al. Root cause analysis of microservices with LLMs and causal inference: How far are we? *Proceedings of the IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024. URL <https://arxiv.org/abs/2408.13729>. arXiv:2408.13729.

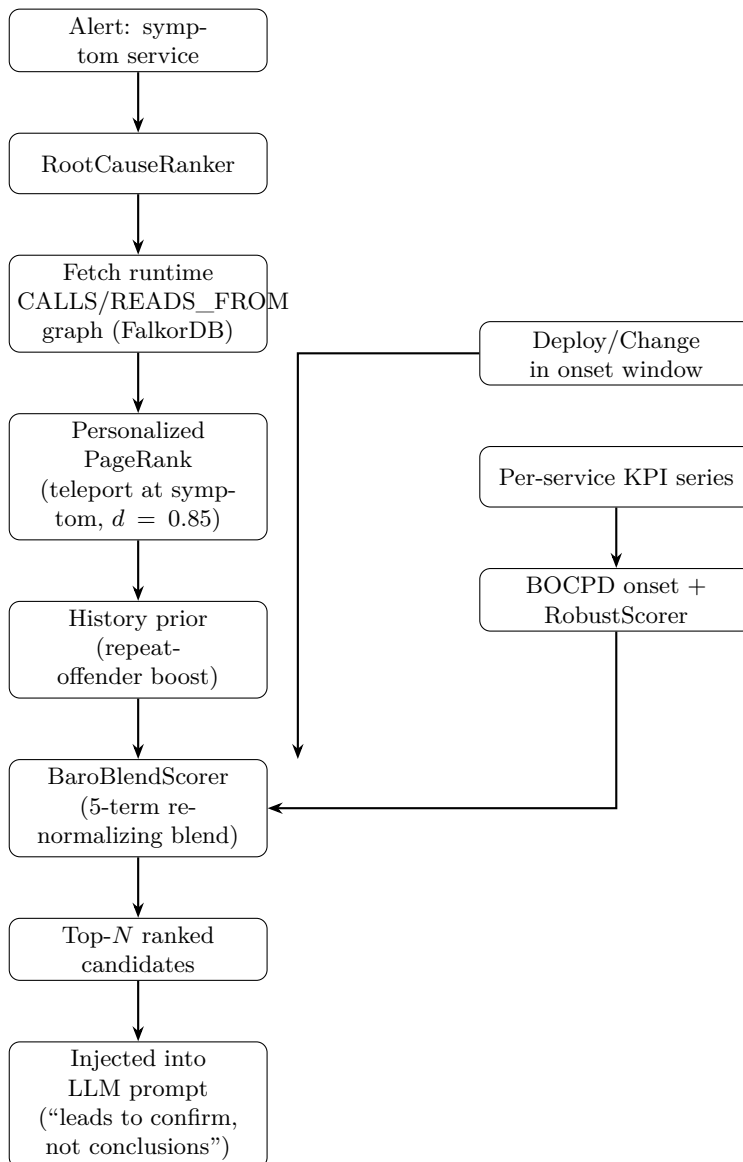


Figure 1: The deterministic pre-LLM scoring pipeline. Every box is pure, training-free, and runs before a single LLM token is spent.